

1- javascript

- a. resultado: false
9007199254740992
- b. o problema é detectado na execução, já que o sistema compila, mas por conta da verificação estrita ele dá um resultado inesperado, ou seja, estávamos esperando um "true" na primeira linha, mas como 0.30 é diferente de 0.3 para o sistema, ele traz "false".

```
console.log(0.1 * 3 === 0.3);
```

```
console.log(9007199254740993);
```

2- python

- a. resultado: 4 6
- b. não possui erro no código, ele é compilado como esperado e executado como esperado. o len(p) lê quantos caracteres a string possui, ou seja, maçã = 4, já o len(p.encode()) pega a quantidade de bytes de caractere e traz para o usuário (m= 1, a= 1, ç= 1, ã= 1)

```
1 p = "maçã"
2
3 print(len(p), len(p.encode()))
4
```

3- go

- a. resultado: 0
- b. não possui erro no código, ele funciona exatamente como deveria funcionar, ao atribuir o valor 255 a variável (o máximo que ela consegue declarar), ao adicionar + 1 (b++) ocorre o overflow, assim resultando em 0

```
1 package main
2 import "fmt"
3
4 func main() {
5     var b byte = 255
6     b++
7     fmt.Println(b)
8 }
```

4- java

- a. resultado: 0
- b. o código erro. A linha v[0] lê o valor armazenado no primeiro índice do array (ou seja, 0), já a linha v[v.length - 1] calcula dinamicamente a última posição válida com base no tamanho total (ou seja, 3 - 1 = 2) e traz o valor contido nela para o usuário (posição 0 = 0, posição 1 = 0, posição 2 = 0).

```
public class Main {
    public static void main() {
        int[] v = new int[3];

        System.out.println(v[0]);

        System.out.println(v[3]);
    }
}
```

5- rust

- a. resultado: erro
- b. o código gera um erro de compilação por design devido ao sistema de propriedade. a linha `let t = s;` realiza um move, o que significa que o rust transfere a posse dos dados da memória de `s` para `t`, invalidando a variável `s` imediatamente. ao tentar ler `s` na linha do `println!`, o compilador bloqueia a execução para garantir a segurança da memória, acusando o erro de "use of moved value".

```
fn main() {  
    let s = String::from("oi");  
    let t = s;  
  
    println!("{}", s, t);  
}
```

6- c

- a. resultado: 1065353216
- b. o código não possui erro de compilação e roda normalmente, mas exibe um valor inesperado por causa do compartilhamento de memória. a linha `u.f = 1.0f;` armazena o número no formato de ponto flutuante, e a linha `u.i` lê a representação binária exata desse float convertida diretamente para um número inteiro.

```
#include <stdio.h>  
int main()  
{  
    union { int i; float f; } u;  
    u.f = 1.0f;  
  
    printf("%d\n", u.i);  
}
```